Developer Guide

Contributing, Development Environment and Code Architecture

Contributing, Development Environment and Code Architecture

Category: Technical Documentation
Prerequisite: Paper 02, Paper 05, Paper 12

1. Introduction for Contributors

WhyCremisi is an open-source (MIT) project and welcomes contributions from audio developers, sound engineers, and AI enthusiasts.

1.1 CODE OF CONDUCT

All contributors must adhere to a code of conduct based on respect, inclusivity, and technical collaboration. We do not tolerate discrimination, harassment, or antisocial behaviour.

1.2 GETTING STARTED

```
1. Read Papers 01-07 to understand the vision
2. Set up the development environment (Section 2)
3. Find an issue labelled "good first issue"
4. Comment on the issue to assign it to yourself
5. Fork → Branch → Commit → PR
```

1.3 ISSUE TRACKER

We use GitHub Issues. Label taxonomy:

LABEL	MEANING
`bug`	Confirmed bug
`enhancement`	New feature
`good first issue`	Suitable for new contributors
`help wanted`	Assistance requested
`plugin-db`	Plugin database related
`daw-integration`	DAW integration related
`ui`	React frontend related
`docs`	Documentation

— 1.4 DEVELOPER FORUM

[NOTE] The #dev channel on the WhyCremisi Discord server is the primary place for technical discussions, architecture questions, and informal code reviews. All significant technical decisions are documented in GitHub Discussions.

• 2. Development Environment

— 2.1 MINIMUM REQUIREMENTS

COMPONENT	VERSION	NOTES
`macOS`	12.7+	Xcode 15.0+ required
`Windows`	10 22H2+	VS 2022 17.8+
`Linux`	Ubuntu 22.04+	GCC 13+ or Clang 16+
`JUCE`	8.0.12	Git submodule
`CMake`	3.28+	Build system
`Node.js`	18+	Frontend toolchain
`npm`	9+	Package manager

— 2.2 INITIAL SETUP

```
# Clone the repository
git clone https://github.com/whycremisi/whycremisi.git
cd whycremisi

# Initialise submodules (JUCE, nlohmann-json, oscpack)
git submodule update --init --recursive

# C++ build (macOS)
cmake -B build -G Xcode -DCMAKE_BUILD_TYPE=Debug
cmake --build build

# Frontend build
cd webview-ui
npm install
npm run dev    # development with HMR
```

— 2.3 RECOMMENDED IDES

Platform	IDE	Configuration
macOS	Xcode 15+	cmake -G Xcode → .xcodeproj
All	VS Code	CMake extension + clangd
All	CLion	Open CMakeLists.txt directly
Windows	Visual Studio	cmake -G "Visual Studio 17 2022"

— 2.4 AUTOMATIC FORMATTING

[NOTE] The project uses `.editorconfig` and `.clang-format`. Ensure your IDE respects these. For VS Code, install the `EditorConfig for VS Code` and `clangd` extensions.

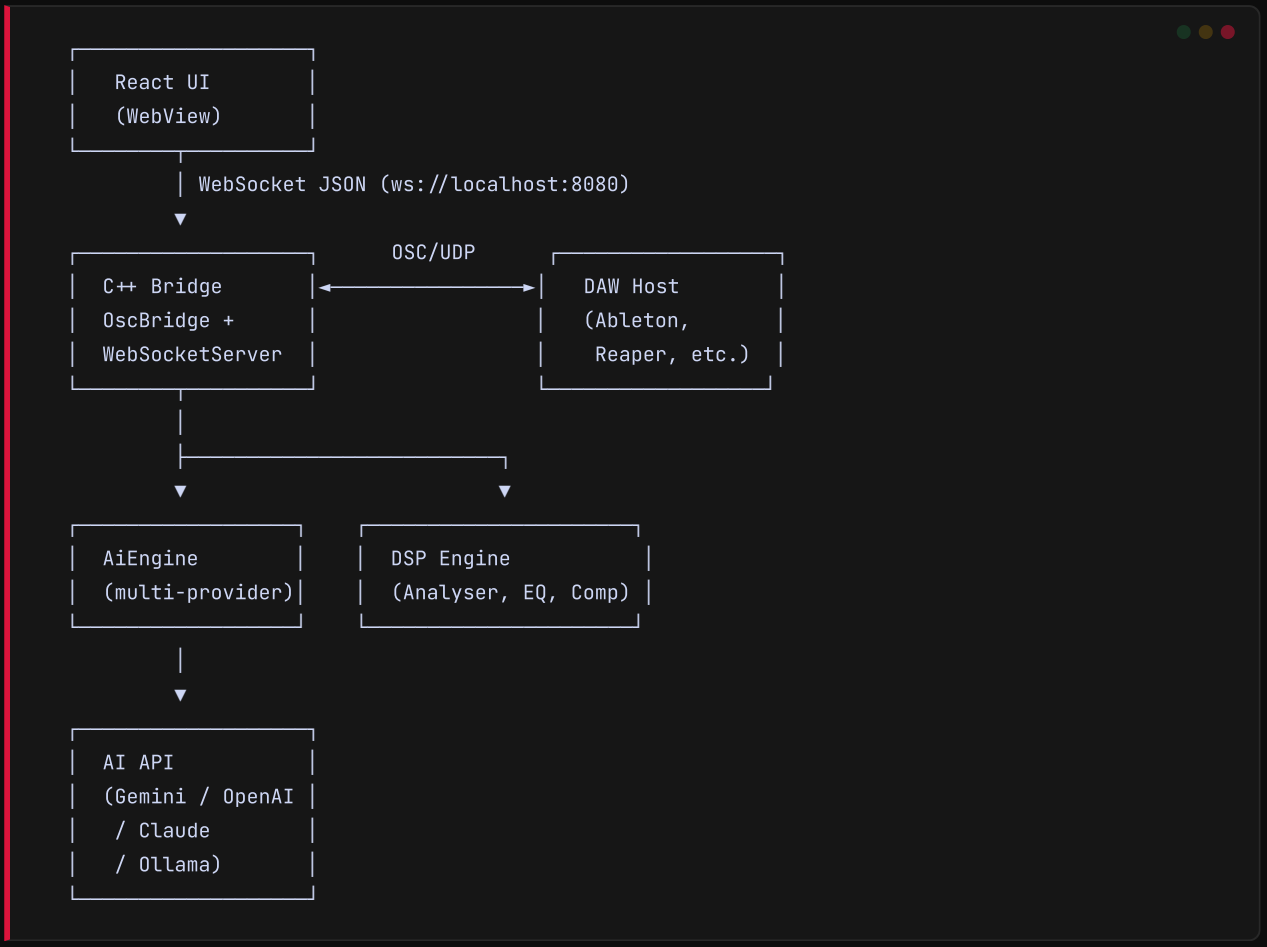
```
# .editorconfig (excerpt)
root = true

[*]
indent_style = space
indent_size = 4
end_of_line = lf
charset = utf-8
trim_trailing_whitespace = true
insert_final_newline = true

[*.{js,jsx,ts,tsx,css,json}]
indent_size = 2
```

3. Code Architecture

3.1 DIRECTORY STRUCTURE

```
WhyCremisi/
├── CMakeLists.txt                  # C++ build root
├── src/
│   ├── core/                       # Main JUCE plugin
│   │   ├── PluginProcessor.h/cpp   # AudioProcessor
│   │   ├── PluginEditor.h/cpp      # AudioProcessorEditor
│   │   ├── ParameterMapper.h/cpp   # Parameter ↔ VST3 mapping
│   │   └── PluginChain.h/cpp       # Internal plugin chain
│   ├── bridge/                     # Communication bridge
│   │   ├── OscBridge.h/cpp         # OSC + UDP server
│   │   ├── WebSocketServer.h/cpp   # WebSocket server
│   │   └── WebViewBridge.h/cpp     # Native WebView bridge
│   ├── ai/                         # AI engine
│   │   ├── AiEngine.h/cpp          # AI orchestrator
│   │   ├── GeminiProvider.h/cpp    # Google Gemini provider
│   │   ├── OpenAIProvider.h/cpp    # OpenAI provider
│   │   └── OllamaProvider.h/cpp    # Local Ollama provider
│   ├── dsp/                        # Audio processing
│   │   ├── Analyzer.h/cpp          # FFT, LUFS, RMS, peak
│   │   ├── Compressor.h/cpp        # Compression
│   │   ├── EQBand.h/cpp            # Parametric filters
│   │   └── DSPEngine.h/cpp         # DSP orchestrator
│   ├── daw/                        # DAW abstraction
│   │   ├── IDawHandler.h           # Virtual interface
│   │   ├── AbletonDawHandler.h/cpp # Ableton Live
│   │   ├── ReaperDawHandler.h/cpp  # Reaper
│   │   └── DawDetector.h/cpp       # Active DAW detection
│   └── agent/                      # Agent system
│       ├── AgentWorkspace.h/cpp     # Identity orchestrator
│       ├── PersonalityCore.h/cpp    # Tone, style, preferences
│       ├── AgentSoul.h/cpp         # Evolutionary memory
│       └── AgentUser.h/cpp         # User profile
├── webview-ui/
│   └── src/
│       ├── App.jsx                 # React root
│       ├── whycremisi-bridge.js    # WebSocket client
│       ├── components/             # Reusable UI components
│       │   ├── BotFace.jsx         # Animated mascot
│       │   ├── BoxChat.jsx         # Contextual box system
│       │   ├── SessionPanel.jsx    # Session panel
│       │   └── SetupScreen.jsx     # Initial setup screen
│       ├── boxes/                  # AI response UI boxes
│       │   ├── EqBox.jsx           # EQ visualiser
│       │   ├── CompressorBox.jsx   # Compressor visualiser
│       │   ├── LevelBox.jsx        # Level meter
│       │   └── SpectrogramBox.jsx  # Spectrogram
│       └── hooks/                  # Custom React hooks
│           ├── useWebSocket.js     # WebSocket hook
│           ├── useDawState.js      # DAW state hook
│           └── useAiStream.js      # AI streaming hook
├── scripts/                        # Utilities
│   ├── validate-plugin.js          # Plugin DB validator
│   └── generate-stubs.js           # Stub generator
└── docs/                           # Extra documentation
```

— 3.2 DATA FLOW



Legend:

DIRECTION	PROTOCOL	DATA
UI → C++	WebSocket JSON	`ai.prompt`, `daw.command`, `plugin.control`
C++ → UI	WebSocket JSON	`ai.stream`, `daw.transport`, `daw.meter`, `plugin.state`
C++ → DAW	OSC / UDP	`transport/play`, `track/volume`, `plugin/param`
DAW → C++	OSC / UDP	`transport/position`, `meter/level`, `track/info`
C++ → AI	HTTP (REST/SSE)	Enriched prompt with session context
AI → C++	HTTP (SSE stream)	Chunked response with tool calls

4. Code Conventions

— 4.1 C++ (JUCE / STANDARD)

ENTITY	CONVENTION	EXAMPLE
Classes	PascalCase	`class OscBridge`
Methods	camelCase	`void processBlock()`
Variables	camelCase	`int sampleRate`
Constants	kPrefixedCamelCase	`constexpr int kMaxBufferSize`
Enum	PascalCase + k prefix	`enum class kState`
Files	PascalCase	`AgentWorkspace.cpp`
Indentation	4 spaces (no tab)	—
Namespace	lowercase	`namespace whycremisi`

— 4.2 REACT / JAVASCRIPT

ENTITY	CONVENTION	EXAMPLE
Components	PascalCase	`function EqBox()`
Functions	camelCase	`const handleStream = ()`
Variables	camelCase	`const wsClient`
Constants	UPPER_SNAKE	`const WS_PORT = 8080`
Component files	PascalCase	`BotFace.jsx`
Utility files	camelCase	`whycremisi-bridge.js`
Indentation	2 spaces (no tab)	—

— 4.3 COMMENTS AND DOCUMENTATION

```
Golden rule:
  • Code comments → ENGLISH
  • Public documentation → ITALIAN (target market)
  • Header/copyright → ENGLISH
```

```
// Code comments in English
void processBlock(AudioBuffer<float>& buffer, MidiBuffer& midi)
{
    // Apply forward gain compensation
    float makeupGain = MathUtil::gainToLinear(3.0f);
    buffer.applyGain(makeupGain);
}
```

```
function EqBox({ data }) {
  // Format frequency band data for display
  const bands = data.bands.map((b) => ({
    freq: `${b.frequency}Hz`,
    gain: b.gain.toFixed(1),
    q: b.q.toFixed(1),
  }));
}
```

— 4.4 GENERAL STYLE

[NOTE] All C++ code must pass `clang-format` with the project's `.clang-format` file. React code must pass ESLint with the shared configuration. PRs that violate formatting are flagged automatically by CI.

• 5. Adding a New Plugin to the Database

— 5.1 PROCESS

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│  1. Open plugins.json in docs/plugins/                   │
│  2. Add entry following Paper 13 schema                  │
│  3. Fill all fields: id, name, manufacturer,             │
│     parameters (with VST3/AU IDs), presets, categories   │
│  4. Run validation                                       │
│  5. Create PR with label "plugin-db"                     │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

5.2 EXAMPLE ENTRY

```
{
  "id": "fabfilter-pro-q-3",
  "name": "Pro-Q 3",
  "manufacturer": "FabFilter",
  "format": ["VST3", "AU", "AAX"],
  "categories": ["EQ", "Dynamic EQ"],
  "parameters": [
    {
      "id": "band1_freq",
      "name": "Band 1 Frequency",
      "vst3Id": 0,
      "range": { "min": 10, "max": 30000, "unit": "Hz" },
      "default": 1000
    },
    {
      "id": "band1_gain",
      "name": "Band 1 Gain",
      "vst3Id": 1,
      "range": { "min": -30, "max": 30, "unit": "dB" },
      "default": 0
    },
    {
      "id": "band1_q",
      "name": "Band 1 Q",
      "vst3Id": 2,
      "range": { "min": 0.1, "max": 100, "unit": "" },
      "default": 1.0
    }
  ],
  "presets": [
    { "name": "Vocal Presence", "params": { "band1_freq": 3000, "band1_gain": 2.5, "band1_q": 2.0 } }
  ]
}
```

5.3 VALIDATION

```
node scripts/validate-plugin.js docs/plugins/fabfilter-pro-q-3.json
# Expected output: ✅ Plugin fabfilter-pro-q-3 validated successfully
```

6. Adding a New DAW Command

6.1 PROCESS

```
┌──────────────────────────────────────────────────────┐
│                                                      │
│  1. Add virtual method to IDawHandler                │
│  2. Implement in AbletonDawHandler and ReaperDawHandler
│  3. Add tool definition in Orchestrator              │
│  4. Map OSC command ↔ WebSocket message              │
│  5. Test with real DAW (Ableton or Reaper)           │
│                                                      │
└──────────────────────────────────────────────────────┘
```

— 6.2 EXAMPLE: "QUANTIZE CLIP" COMMAND

Step 1 — Interface:

```
// src/daw/IDawHandler.h
class IDawHandler
{
public:
    virtual ~IDawHandler() = default;

    virtual void play() = 0;
    virtual void stop() = 0;
    virtual void setTrackVolume(int trackIndex, float volume) = 0;

    // ✨ New command
    virtual void quantizeClip(int trackIndex, int clipIndex, float swing) = 0;
};
```

Step 2 — Reaper Implementation:

```
// src/daw/ReaperDawHandler.cpp
void ReaperDawHandler::quantizeClip(int trackIndex, int clipIndex, float swing)
{
    // Reaper OSC API: /track/{idx}/item/{clipIdx}/quantize
    oscSender->sendMessage(
        osc::OutboundPacketStream(buffer, kBufferSize)
            << osc::BeginMessage("/track")
            << trackIndex
            << osc::BeginMessage("/item")
            << clipIndex
            << osc::BeginMessage("/quantize")
            << swing
            << osc::EndMessage
    );
}
```

Step 3 — Tool Definition:

```
// In AgentWorkspace::getTools()
Tool quantizeTool{
    .name = "daw_quantize_clip",
    .description = "Quantize a MIDI clip on the specified track",
    .parameters = {
        {"trackIndex", Type::kInteger, "Track index (0-based)"},
        {"clipIndex",  Type::kInteger, "Clip index (0-based)"},
        {"swing",      Type::kFloat,   "Swing amount (0.0-1.0)"}
    }
};
```

[NOTE] The WebSocket handler must translate incoming messages (`{ type: "daw.command", payload: { command: "quantize_clip", ... } }`) into the corresponding method call on `IDawHandler` . The mapping lives in `OscBridge::handleCommand()` .

---

7. Adding a New Box UI

— 7.1 PROCESS

```
┌──────────────────────────────────────────────────────────┐
│                                                          │
│  1. Create webview-ui/src/boxes/NomeBox.jsx              │
│  2. Follow BoxContext + motion (Framer Motion) pattern   │
│  3. Register the new box in BoxChat.jsx                  │
│  4. Test with mock data                                  │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

— 7.2 EXAMPLE: METERBOX

```jsx
// webview-ui/src/boxes/MeterBox.jsx
import { motion } from 'framer-motion';
import { useBoxContext } from '../hooks/useBoxContext';

export default function MeterBox({ data }) {
  const { onAction } = useBoxContext();

  return (
    <motion.div
      className="rounded-xl bg-zinc-800 p-4"
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
    >
      <h3 className="text-sm font-medium text-zinc-400">Level Meter</h3>
      <div className="mt-2 flex items-end gap-1">
        {data.levels.map((level, i) => (
          <div
            key={i}
            className="w-4 rounded-t bg-emerald-500 transition-all"
            style={{ height: `${level * 100}%` }}
          />
        ))}
      </div>
      <div className="mt-2 flex justify-between text-xs text-zinc-500">
        <span>L</span>
        <span>{data.peak.toFixed(1)} dB</span>
        <span>R</span>
      </div>
    </motion.div>
  );
}
```

7.3 REGISTRATION IN BOXCHAT

```
// webview-ui/src/components/BoxChat.jsx
import MeterBox from '../boxes/MeterBox';

const BOX_RENDERERS = {
  eq:          EqBox,
  compressor:  CompressorBox,
  level:       MeterBox,       // ⟵ New box registered
  spectrogram: SpectrogramBox,
};

export default function BoxChat({ messages }) {
  return messages.map((msg) => {
    const Renderer = BOX_RENDERERS[msg.type];
    return Renderer ? <Renderer key={msg.id} data={msg.data} /> : null;
  });
}
```

---

8. Testing

8.1 C++ TESTS (JUCE UNITTEST)

```
# Build and run native tests
cmake --build build --target WhyCremisi_UnitTests
./build/WhyCremisi_UnitTests

# Expected output:
# [OK] 32 tests passed (4 test suites)
```

8.2 FRONTEND TESTS (VITEST)

```
cd webview-ui
npm test

# Expected output:
#  PASS  src/__tests__/BoxChat.test.jsx
#  PASS  src/__tests__/useWebSocket.test.js
```

8.3 LINT

```
cd webview-ui
npm run lint

# No output = clean
# ESLint errors block merge
```

[NOTE] CI automatically runs C++ and frontend tests on every PR. The build fails if tests do not pass.

---

9. Pull Request Process

9.1 BRANCH NAMING

PREFIX	USAGE
`feature/`	New feature
`fix/`	Bug fix
`docs/`	Documentation
`refactor/`	Refactoring
`test/`	Test addition/modification
`chore/`	Maintenance (deps, CI, build)

9.2 COMMIT MESSAGE CONVENTION

```
type(scope): description

Types: feat, fix, docs, refactor, test, chore

Examples:
  feat(daw): add quantize clip command
  fix(ui): correct meter bar overflow on narrow screens
  docs(plugin-db): add FabFilter Pro-R2 entry
  refactor(bridge): extract OSC message parser
  test(ai): add streaming timeout edge case
```

9.3 REVIEW AND MERGE

```
┌──────────────────────────────────────────────────┐
│                                                  │
│  1. Fork the repository                          │
│  2. Create branch with correct naming            │
│  3. Commit following convention                  │
│  4. Open PR with appropriate label               │
│  5. At least 1 approval from a maintainer        │
│  6. CI green (test + lint + build)               │
│  7. Squash merge to main                         │
│                                                  │
└──────────────────────────────────────────────────┘
```

10. Release Process

10.1 PRE-RELEASE CHECKLIST

```
□ All C++ tests pass
□ All frontend tests pass
□ npm run lint clean
□ Build verified on macOS, Windows, Linux
□ Documentation updated (related Papers)
□ CHANGELOG.md updated
□ Version bump in CMakeLists.txt (follows Paper 12)
```

10.2 VERSION BUMP

```
# CMakeLists.txt
project(WhyCremisi VERSION 0.5.0)  # ← e.g. 0.4.0 → 0.5.0
```

10.3 TAG AND GITHUB RELEASE

```
git tag -a v0.5.0 -m "v0.5.0 — DAW quantization, new plugin DB entries"
git push origin v0.5.0
```

[NOTE] The GitHub Release is created automatically by CI on tag push, with pre-built assets for all three platforms.